

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание игры в стиле ООП

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

ВВЕДЕНИЕ

Цель: разработка модульной и расширяемой архитектуры пошаговой тактической игры с применением современных принципов объектно-ориентированного программирования на C++, обеспечивающей гибкость модификации игровой логики и эффективное управление ресурсами.

Задачи:

1. Создать класс(ы) игры, реализующий основной игровой цикл, которому передаются команды от пользователя;
2. Реализовать систему сохранения и загрузки игры;
3. Добавить обработку исключительных ситуаций для загрузки/сохранения;
4. Реализовать переход на следующий уровень после прохождения текущего уровня;
5. Реализовать прокачку игрока при переходе между уровнями.

ОПИСАНИЕ СТРУКТУРЫ КОДА

Структура *LevelAttributes* хранит параметры уровня, задаваемые перед его началом.

Поля:

- *std::vector<spellType> spells* — набор доступных игроку заклинаний.
- *int enemyHP* — здоровье врага.
- *int playerHP* — здоровье игрока.
- *int fieldWidth* — ширина игрового поля.
- *int fieldLength* — длина игрового поля.
- *int spellDamageKoef* — коэффициент силы заклинаний.
- *int playerDamageKoef* — коэффициент урона игрока.
- *int playerImproveHP* — поле, отслеживающее улучшение здоровья игрока.
- *int enemyDamage* — урон врага.
- *int goalMoves* — лимит ходов для победы.
- *int goalScore* — требуемое количество убийств врагов для победы.

Структура *SaveData* содержит полную информацию о переменных параметрах уровня, необходимых для сохранения и загрузки игры.

Поля:

- *int fieldLength* — длина игрового поля.
- *int fieldWidth* — ширина игрового поля.
- *std::vector<int> cellTypes* — типы клеток поля.
- *std::vector<int> cellCharacters* — персонажи, находящиеся в клетках поля.
- *std::vector<int> cellDamages* — урон клеток поля (для ловушек).
- *int playerHealth* — текущее здоровье игрока.

- *int playerCombatType* — тип атаки игрока (ближняя/дальняя).
- *int playerDamage* — значение урона игрока.
- *int playerDamageKoef* — коэффициент урона игрока.
- *int playerImproveHP* — поле, отслеживающее улучшение здоровья игрока.
- *int coins* — монеты игрока.
- *int score* — счет игрока (количество побежденных врагов).
- *bool playerMoveAbility* — возможность передвижения игрока.
- *std::pair<int,int> playerCoordinates* — позиция игрока на поле.
- *std::vector<int> spellTypes* — сохранённые карточки заклинаний.
- *int spellsKoef* — коэффициент урона заклинаний.
- *int enemyHealth* — здоровье врага.
- *int enemyDamage* — урон врага.
- *std::pair<int,int> enemyCoordinates* — позиция врага на поле.
- *std::pair<int,int> towerCoordinates* — координаты вражеской башни.
- *int moves* — количество сделанных ходов.
- *int gameCondition* — состояние игры (меню, игра, победа, поражение, улучшение).
- *int goalMoves* — максимальное количество ходов для прохождения уровня.
- *int goalScore* — количество врагов, которых нужно победить для прохождения уровня.
- *int cellSize* — сохранённый размер клетки.
- *int hash* — хэш сохраненной структуры.

Класс *SaveManager* отвечает за загрузку и сохранение игры в файл формата *JSON*.

Поля:

- *std::string filePath* — хранит путь к файлу, в котором будет сохранена игра и из которого будут выгружаться данные для восстановления сохраненной игры.

Методы:

1. *bool saveToJson(SaveData saveData)* — метод, сохраняющий данные игры в *json-файл*.

Принимаемый аргумент:

SaveData saveData — структура с данными уровня, который необходимо сохранить.

Возвращаемое значение:

true, если сохранение произошло успешно, иначе *false*.

2. *bool loadFromJson(SaveData& saveData)* — метод, выгружающий в структуру сохраненные в *json-файле* данные игры.

Принимаемый аргумент:

SaveData& saveData — ссылка на структуру, в которую будут записаны сохраненные данные.

Возвращаемое значение:

true, если сохранение произошло успешно, иначе *false*.

3. *int makeHash(SaveData data)* — метод, формирующий хэш для всех полей сохраненных данных в целях их защиты от изменения извне.

Принимаемый аргумент:

SaveData data — структура с записанными данными игры.

Возвращаемое значение:

int hash — хэш, сформированный от всех полей структуры сохраненных данных.

4. *int mix(int hash, int value)* — метод, позволяющий производить постепенное шифрование: берется предыдущий хэш, очередное поле

структуры сохранений и с помощью некоторой функции на основе полученных аргументов вычисляется новый хэш.

Принимаемые аргументы:

int hash — предыдущий хэш.

int value — значение поля структуры с сохранениями.

Возвращаемое значение:

int newHash — новый хэш.

5. *bool checkSaveData(SaveData data)* — метод, пересчитывающий хэш полученной структуры и сравнивающий его с тем, который в ней сохранен.

Принимаемые аргумент:

SaveData data — структура с сохраненными данными.

Возвращаемое значение:

true, если хэши совпали, иначе *false*.

Класс *GameManager* хранит главные игровые сущности и контролирует их взаимодействие.

Поля:

- *Player player* — объект игрока.
- *Enemy enemy* — объект врага.
- *Field field* — объект игрового поля.
- *EnemyTower tower* — объект вражеской башни.
- *int moves* — текущее количество ходов игрока.

Методы:

1. *GameManager()* - конструктор по умолчанию, создает объекты сущностей и инициализирует начальные значения для новой игры (игрок со здоровьем 10, враг со здоровьем 5 и уроном 1, поле размером 15x15

клеток, вражеская башня с характеристиками по умолчанию, количество затраченных ходов 0).

2. *GameManager(LevelAttributes levelParameters)* — конструктор с параметрами для следующего уровня, создает сущности и устанавливает им заданные параметры. Внутри добавляет заклинания в руку игрока и меняет коэффициент его урона.

Принимаемый аргумент:

LevelAttributes levelParameters — структура с параметрами следующего уровня.

3. *Player getPlayer() const* — метод получения объекта игрока.

Возвращаемое значение:

Player player — текущий объект игрока.

4. *void setPlayer(Player newPlayer)* — метод установки нового объекта игрока.

Принимаемый аргумент:

Player newPlayer — новый объект игрока.

5. *Enemy getEnemy() const* — метод получения объекта врага. Используется для получения информации о здоровье и для графического отображения.

Возвращаемое значение:

Enemy enemy — текущий объект врага.

6. *Field getField() const* — метод получения объекта поля. Используется для получения информации о размерах поля и для графического отображения.

Возвращаемое значение:

Field field — текущий объект поля.

7. *std::pair<int, int> getTowerCoords() const* — метод получения координат вражеской башни.

Возвращаемое значение:

std::pair<int, int> coords — текущие координаты вражеской башни.

8. *int getMoves() const* — метод получения текущего количества затраченных ходов.

Возвращаемое значение:

int currentMoves — количество ходов.

9. *void setMoves(int newValue)* — метод установки нового количества ходов.

Используется в классе *Game* при ударах игрока и переключении типа боя.

Принимаемый аргумент:

int newValue — новое количество ходов.

10. *void placeCharacter(character person)* — метод размещения персонажа на поле. Случайным образом выбирает свободную клетку и ставит на ней персонажа.

Принимаемый аргумент:

character person — персонаж для размещения.

11. *bool isEnemyInRange(int range)* — метод, который позволяет определить, находится ли враг на заданном расстоянии. Используется для атаки врага в зависимости от типа боя игрока.

Принимаемый аргумент:

int range — проверяемое расстояние.

Возвращаемое значение:

true, если враг находится на заданном расстоянии, иначе *false*.

12. *bool movePlayer(char direction)* — метод, отвечающий за перемещение игрока в заданном направлении.

Принимаемый аргумент:

char direction — направление

Возвращаемое значение:

true, если игрок переместился, иначе *false*.

13. *void moveEnemy()* — метод, отвечающий за перемещение врага. В зависимости от положения игрока выбирается оптимальная клетка, в которую двигается враг.

14. *void attackEnemy()* - метод, где игрок атакует врага.

15. *void attackPlayer()* - метод, где враг атакует игрока.

16. *bool applySpell(int spellIndex, std::pair<int, int> target)* — метод применения заклинания.

Принимаемые аргументы:

int spellIndex — индекс заклинания в руке.

std::pair<int, int> target — координаты цели, к которой будет применено заклинание.

Возвращаемое значение:

true, если заклинание применено, иначе *false*.

17. *void buySpell(int koef)* — метод для покупки заклинания.

Принимаемый аргумент:

int koef — коэффициент урона заклинания.

18. *bool towerAttack(Player& target)* — метод, где башня атакует игрока.

Принимаемый аргумент:

Player& target — ссылка на объект игрока.

Возвращаемое значение:

true, если башня ударила, иначе *false*.

19. *SaveData convertToSaveData()* — метод, сохраняющий данные игры в структуру для их последующего сохранения в файл.

Возвращаемое значение:

SaveData data — структура с записанными данными игры.

20. *void unpackSaveData(SaveData data)* — метод, распаковывающий и применяющий сохраненные данные к текущей игре.

Принимаемые аргументы:

SaveData data — структура, из которой будут распакованы данные.

Класс *Game* реализует взаимодействие пользователя с игрой через клавиатуру и мышь, а также создает окно, в котором отображается игра.

Поля:

- *sf::RenderWindow window* — окно.
- *GameManager gameManager* — объект игры.
- *GameStatus gameStatus* — текущее состояние игры.
- *GameGraphics GUI* — графическое отображение.
- *SaveManager saver* — объект для сохранения игры.
- *LevelAttributes levelParameters* — структура с параметрами уровня.
- *bool goToNextLVL* — поле, которое позволяет определить момент перехода на следующий уровень.
- *int windowWidth* — ширина окна.
- *int windowHeight* — высота окна.
- *int selectedSpellCard* — выбранная карточка заклинания.
- *bool waitingForTarget* — поле для ожидания ввода с мыши.
- *bool isTowerAttack* — поле, показывающее атаковала ли вражеская башня.
- *int timeWithoutAttack* — поле, показывающее оставшееся количество ходов, в которые башня не будет атаковать в своем радиусе.
- *int CELL_SIZE* — размер клетки.
- *OverlayType currentOverlayType* — тип слоя для отрисовки радиусов атаки и применения заклинаний.

- *bool showRangeOverlay* — поле, определяющее нужно ли показывать слой отрисовки из пункта выше.

Методы:

1. *Game()* - конструктор по умолчанию. Инициализирует игровой статус, создает игру, объект для ее отрисовки и окно, в котором будет отображаться игровой процесс.
2. *void start()* - метод запуска игры. Размещает на поле игрока и вражескую башню, обновляет размеры клеток и инициализирует карточки заклинаний (для отрисовки), вызывает метод обработки ввода, после которых в зависимости от статуса игры происходит отрисовка.
3. *void checkGameState()* - метод установки состояния игры на победу/поражение.
4. *void saveGame()* - метод сохранения игры. Выгруженные данные по сущностям из *GameManager* и данные о целях уровня и размерах клетки передаются в метод класса *SaveManager*, который сохраняет их в *json-файле*.
5. *void loadGame()* - метод загрузки игры. Вызывает метод класса *SaveManager* для выгрузки данных из файла, затем сохраняет их в структуре параметров уровня и передает на распаковку в *GameManager*.
6. *void handleInput()* - метод обработки ввода с клавиатуры. В зависимости от введенного символа выполняются различные игровые функции. После ввода происходит движение врага и атака вражеской башни.
7. *void handleMenuClick(int mouseX, int mouseY)* — обработка клика мыши в главном меню и при победе/поражении.

Принимаемые аргументы:

int mouseX, int mouseY — координаты клика мыши.

8. *void handleClick(int mouseX, int mouseY)* — обработка клика мыши для выбора цели применения заклинания.

Принимаемые аргументы:

int mouseX, int mouseY — координаты клика мыши.

9. *void nextLevel()* - метод перехода на следующий уровень. Изменяется размер поля, здоровье игрока увеличивается на 5, удаляется половина карточек заклинаний из руки, здоровье и урон врага увеличиваются на 1, также увеличиваются значения целей уровня.

10. *bool checkButton(int mouseX, int mouseY, int btnX, int btnY, int btnWidth, int btnHeight)* — метод проверки клика на кнопку.

Принимаемые аргументы:

int mouseX, int mouseY — координаты клика мыши.

int btnX, int btnY — левая верхняя координата кнопки.

int btnWidth — ширина кнопки.

int btnHeight — высота кнопки.

Возвращаемое значение:

true, если клик попал по кнопке, иначе *false*.

Взаимосвязь классов представлена на UML-диаграмме (см. Приложение).

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы были созданы классы, полностью реализующие игровую логику: класс *GameManager* обеспечивает связь между всеми компонентами системы и управляет их взаимодействием; класс *Game* принимает команды пользователя, связывает их с действиями в игре, а также осуществляет переход между игровыми состояниями; класс *SaveManager* отвечает за загрузку и сохранение игры в *json-файл*. При успешном прохождении текущего уровня предоставляется возможность перехода на следующий уровень с измененными параметрами размеров поля, здоровья и урона врага и игрока, содержимого «руки» и целей уровня. Перед началом следующего уровня добавлен набор различных улучшений характеристик игрока. Соблюдены принципы инкапсуляции, исключено дублирование данных. Программа протестирована и работает корректно.

ПРИЛОЖЕНИЕ

